

ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO



PCS3732 - Laboratório de Processadores

Projeto ARMLator

Integrantes:

João Pedro Bassetti Freitas	13903411
João Pedro Lopes de Sousa Gomes	13680810
Mauro Martins Pereira Junior	13679798

15/Agosto/2025

1. Introdução

O projeto ARMLator consiste no desenvolvimento de um simulador educacional interativo da arquitetura ARM, voltado para fins didáticos no ensino de organização e arquitetura de computadores. A ferramenta tem como objetivo principal facilitar o entendimento do funcionamento interno de processadores ARM, permitindo que usuários acompanhem, passo a passo, a execução de instruções em linguagem Assembly ou binária.

A interface do ARMLator é web e interativa, acessível diretamente pelo navegador sem necessidade de instalação, inspirada em plataformas como o CPULator. A proposta é que o usuário possa escrever seu código, carregar e executar instrução por instrução, observando visualmente as mudanças nos registradores, na memória e no registrador de status CPSR, que contém as flags N (negativo), Z (zero), C (carry) e V (overflow). Essas alterações são destacadas dinamicamente após cada execução, promovendo um ambiente de aprendizado mais intuitivo e visual.

O simulador foi projetado para abranger inicialmente um subconjunto das instruções ARM mais relevantes, como MOV, ADD, SUB, CMP, B, LDR e STR, além de suportar desvios condicionais e operações lógicas e aritméticas básicas. A lógica de simulação, o interpretador de instruções e a interface gráfica foram desenvolvidos de forma modular, visando facilitar a manutenção, expansão e integração dos componentes.

2. Descrição da plataforma e tecnologias utilizadas

A plataforma é dividida em três camadas principais: backend, servidor intermediário e frontend. O backend, escrito em C++, é responsável por interpretar e executar o código Assembly, atualizando os registradores (R0 a R15), a memória simulada de 32 posições e as flags do registrador CPSR (N, Z, C, V) a cada operação. Ele já suporta diversas instruções essenciais como MOV, ADD, SUB, CMP, LDR, STR, BEQ, BNE, MUL, DIV, AND, ORR, EOR, LSL, LSR, entre outras. A lógica interna trata as instruções linha a linha, implementando controle de fluxo com desvios condicionais e manipulação de memória por índices.

```
[enter] para próximo passo, [f] para finalizar execução:
R0 = 0
R1 = 5
R2 = 0
R3 = 0
R4 = 0
R5 = 0
R6 = 0
R7 = 0
R8 = 0
R9 = 0
R10 = 0
R11 = 0
R12 = 0
R13 = 0
R14 = 0
R15 = 0
N = 0
Z = 0
C = 0
V = 0
PC = 2
MEM 0x1000 = 0x00000000
MEM 0x1004 = 0x00000000
MEM 0x1008 = 0x00000000
MEM 0x100C = 0x00000000
MEM 0x1010 = 0x00000000
MEM 0x1014 = 0x00000000
MEM 0x1018 = 0x00000000
MEM 0x101C = 0x00000000
----
[enter] para próximo passo, [f] para finalizar execução:
```

Figura 1: Execução do backend exibindo os valores atualizados dos registradores e flags

Para conectar o simulador ao navegador, foi implementado um servidor intermediário em Node.js, que atua como ponte entre o backend e o frontend. Ele recebe o código do usuário, envia para o backend, executa o binário do simulador e retorna os dados processados para serem exibidos na interface gráfica. Essa separação permite que a lógica de simulação permaneça desacoplada da interface, facilitando futuras atualizações em cada camada.

A interface gráfica foi desenvolvida em React, com foco didático. O usuário pode colar ou digitar o código em Assembly em um editor embutido, carregar o programa e acompanhar a execução por meio de botões como "Carregar", "Passo", "Executar" e "Resetar". A tela exibe a tabela dos 16 registradores, o conteúdo da memória simulada e o estado atual do CPSR.

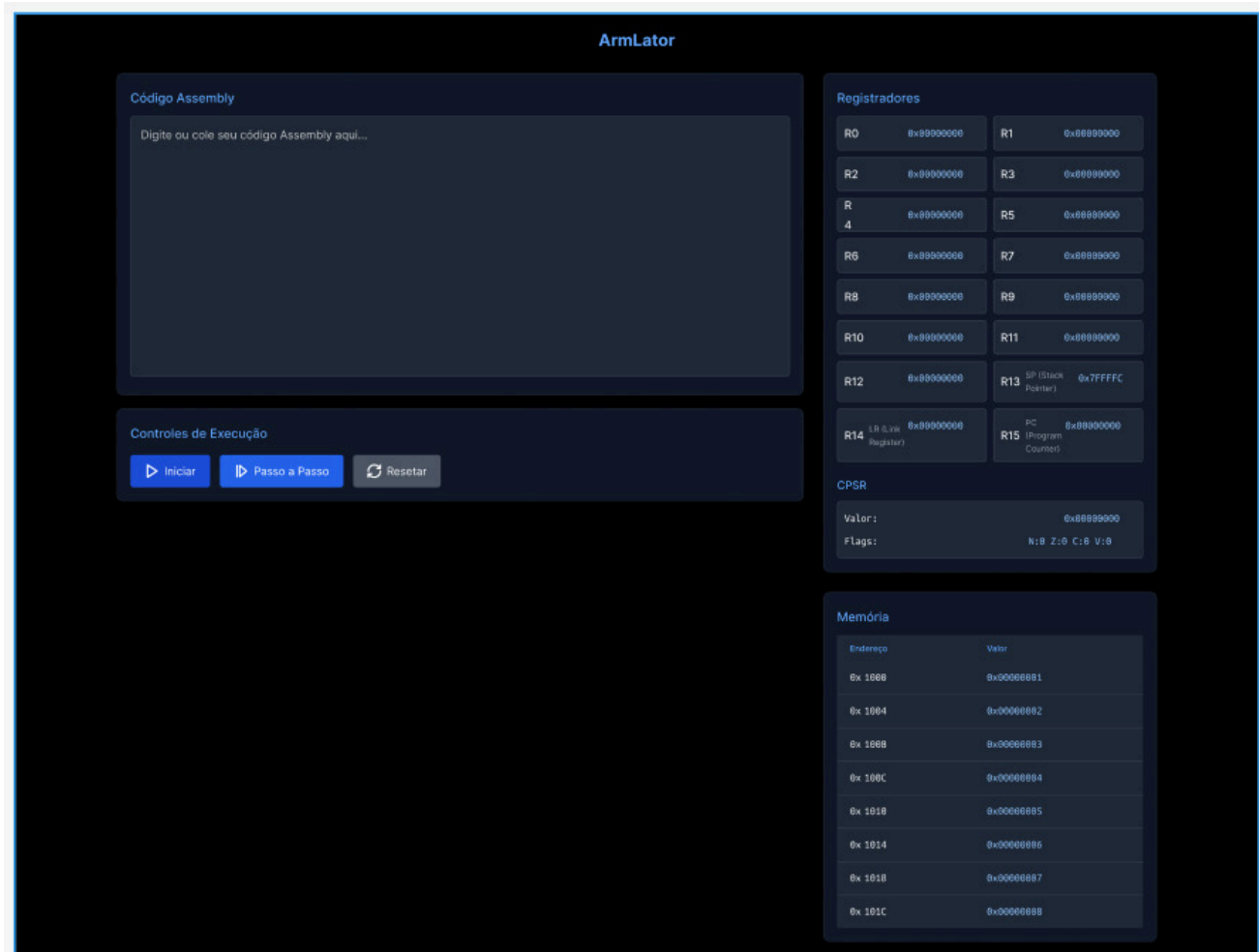


Figura 2: Interface gráfica do ARMLator com editor de código, visualização dos registradores e estado do CPSR.

O parser da plataforma é flexível: ele aceita vírgulas, colchetes (em instruções como LDR R0, [R1]) e comentários iniciados por ;. Instruções inválidas são identificadas com mensagens claras de erro. O interpretador reconhece tanto operandos imediatos quanto registradores, diferenciando operações como MOV R1, #10 e MOV R1, R2.

O ARMLator suporta a execução de algoritmos didáticos clássicos, como cálculo de fatorial, sequência de Fibonacci e contagem de bits, permitindo que os usuários acompanhem programas completos em tempo real, e não apenas instruções isoladas.

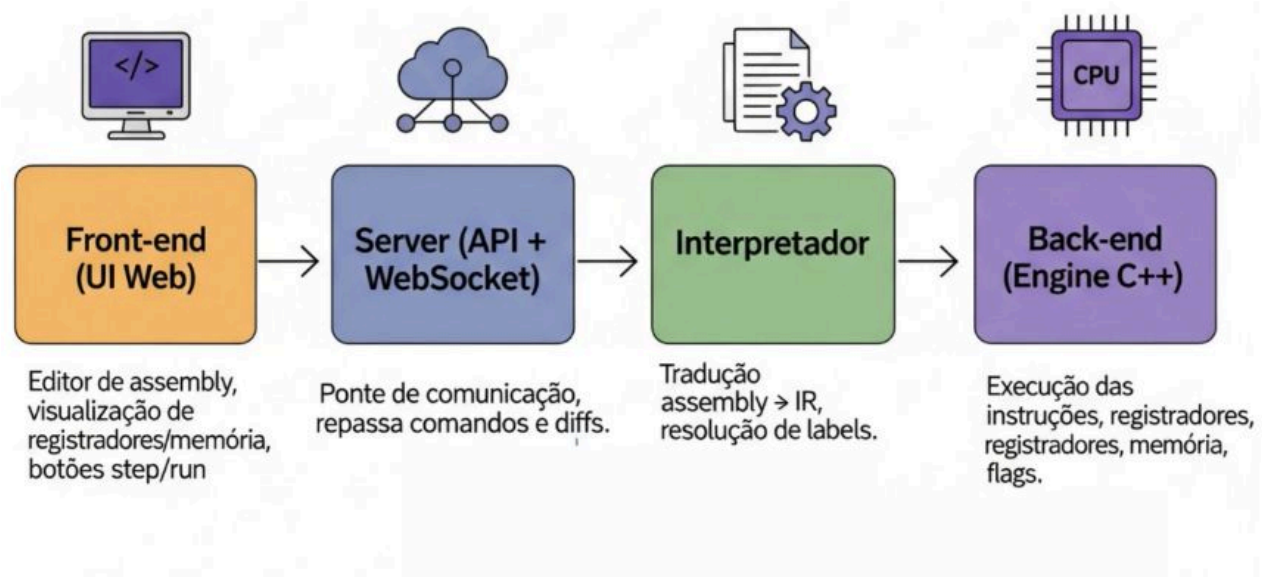


Figura 3: Arquitetura da plataforma ARMLator conectando interface, servidor intermediário e simulador.

3. Implementação Técnica

O ARMLator é estruturado de forma modular, com três componentes principais: o interpretador, responsável por transformar o código assembly em instruções internas; o simulador, que executa essas instruções mantendo o estado de registradores, memória e flags; e o arquivo `main.cpp`, que integra ambos e inicia a execução. Essa separação facilita manutenção e expansão do sistema, além de tornar clara a lógica de execução passo a passo.

O núcleo do simulador utiliza a enumeração **TipInstrucao** e a struct **Instrucao** para representar as instruções e seus operandos:

```

enum class TipoInstrucao {
    NOP, MOV_IMD, MOV_REG, ADD_IMD, ADD_REG, CMP_IMD, CMP_REG,
    BEQ, BNE, LDR, STR, SUB_IMD, SUB_REG, MUL, DIV, AND, ORR, EOR,
    LSL, LSR, B
};

struct Instrucao {
    TipoInstrucao tipo;
    int destino = 0;
    int origem1 = 0;
    int origem2 = 0;
    int imediato = 0;
    int offset = 0;
};

```

O interpretador lê um arquivo de texto contendo o código Assembly e converte cada linha em um objeto Instrucao. Ele ignora espaços em branco e comentários, e distingue instruções com operandos imediatos ou registradores. Por exemplo, a interpretação de MOV com imediato e registrador é feita da seguinte forma:

```

if (opcode == "MOV") {
    std::string dest, val;
    ss >> dest >> val;
    inst.destino = std::stoi(dest.substr(1));
    if (val[0] == '#') {
        inst.tipo = TipoInstrucao::MOV_IMD;
        inst.imediato = std::stoi(val.substr(1));
    } else {
        inst.tipo = TipoInstrucao::MOV_REG;
        inst.origem1 = std::stoi(val.substr(1));
    }
}
}

```

O simulador mantém o estado de 16 registradores, memória simplificada e flags do CPSR (Negative, Zero, Carry e Overflow). As instruções são executadas em um loop, com cada tipo de operação atualizado de acordo com sua lógica:

```

case TipoInstrucao::ADD_IMD: {
    int64_t res = registradores[inst.origem1] + inst.imediato;
    registradores[inst.destino] = static_cast<int>(res);
    atualizaFlagsADD(registradores[inst.origem1], inst.imediato, res);
    pc++;
    break;
}

```

As flags são atualizadas por funções auxiliares que implementam a lógica do ARM de forma simplificada. Por exemplo, a função que atualiza as flags após uma adição considera overflow, carry, zero e sinal:

```

static inline void atualizaFlagsADD(int32_t a, int32_t b, int64_t res64) {
    int32_t r = static_cast<int32_t>(res64);
    flagN = (r & 0x80000000) != 0;
    flagZ = (r == 0);
    flagC = (static_cast<uint64_t>(a) + static_cast<uint64_t>(b)) > 0xFFFFFFFFFull;
    flagV = ((~(a ^ b)) & (a ^ r) & 0x80000000) != 0;
}

```

O main.cpp integra os módulos, carregando o arquivo de programa e executando as instruções:

```

int main() {
    std::string caminho = "program.txt";
    auto instrucoes = interpretar_arquivo(caminho);
    executar_programa(instrucoes);
    return 0;
}

```

Além disso, o servidor intermediário, implementado em Node.js, atua como ponte entre a interface gráfica e o simulador em C++. Ao receber o código Assembly enviado pelo frontend, o servidor o salva em um arquivo program.txt e executa o binário armlator.exe utilizando o módulo child_process. A execução é feita de forma síncrona, e toda a saída padrão gerada pelo simulador, que inclui o estado dos registradores, memória e flags, é capturada e enviada de volta para o frontend via HTTP.

4. Caso de uso

Com o objetivo de testar a plataforma, foi utilizado o código em assembly desenvolvido pelo grupo na aula 2 que, dado um número natural n , retorna o n -ésimo elemento da sequência de Fibonacci. O número n é colocado no registrador R0 (nesse caso, o número 10 em decimal) e o resultado final é armazenado no registrador R1. O código utilizado, colocado no campo de escrita da plataforma ArmLator, pode ser visto na Figura 4.

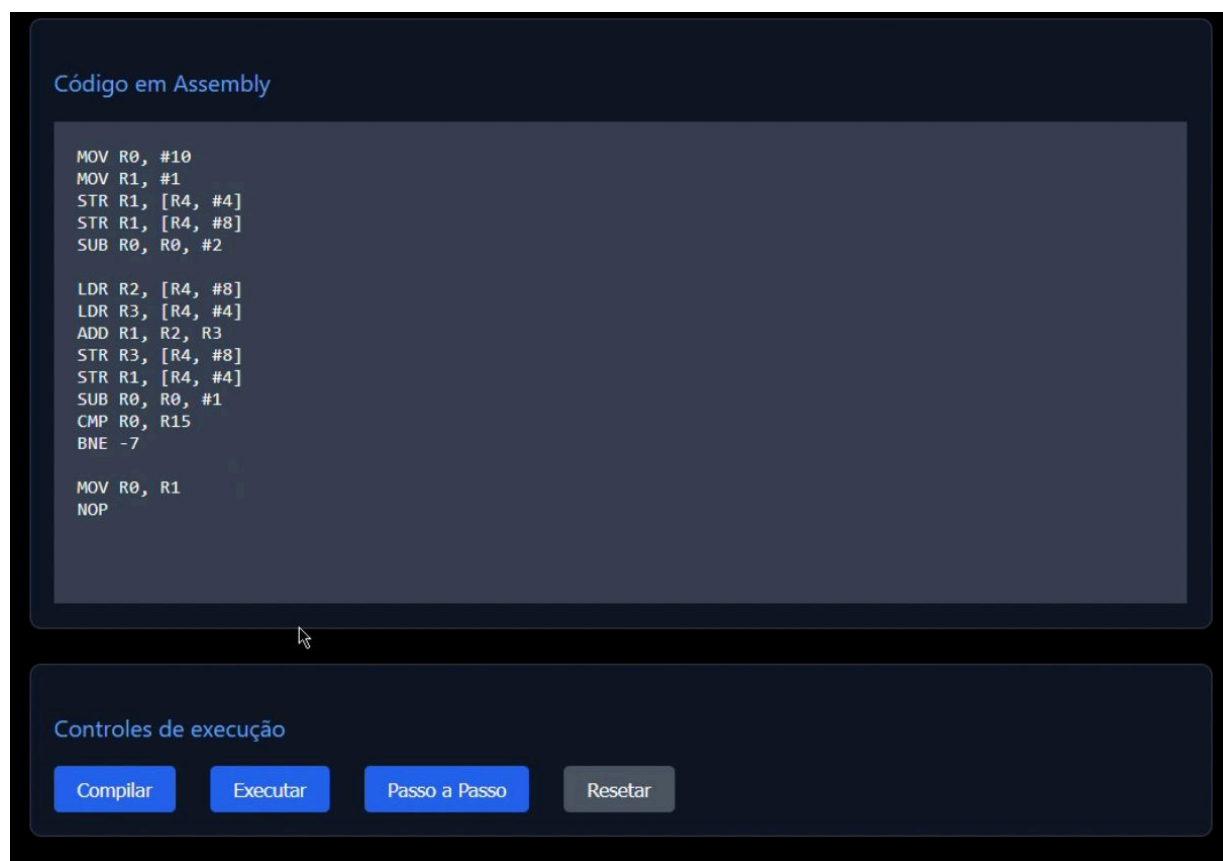
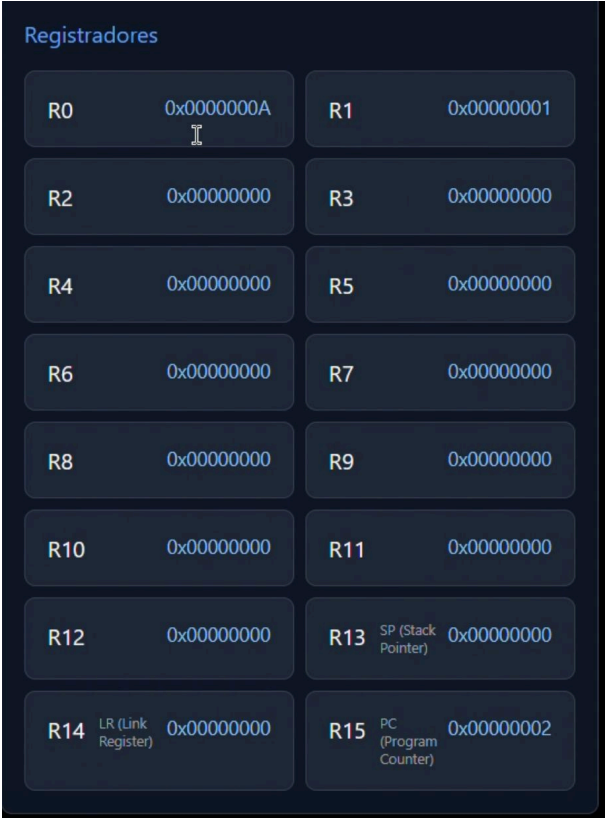


Figura 4: Código da sequência de Fibonacci usado como teste da plataforma ArmLator

Após clicar em “compilar”, são executadas as 2 primeiras instruções, atualizando o valor da memória e do conteúdo dos registradores, conforme a figura abaixo.



R0	0x0000000A	R1	0x00000001
R2	0x00000000	R3	0x00000000
R4	0x00000000	R5	0x00000000
R6	0x00000000	R7	0x00000000
R8	0x00000000	R9	0x00000000
R10	0x00000000	R11	0x00000000
R12	0x00000000	R13	SP (Stack Pointer) 0x00000000
R14	LR (Link Register) 0x00000000	R15	PC (Program Counter) 0x00000002

Figura 5: Valor dos registradores exibidos na plataforma após compilação do código

A partir desse ponto, o usuário tem a opção de clicar em “passo a passo”, acompanhando o valor da memória e do conteúdo dos registradores (bem como as flags) após a execução de cada instrução ou clicar em “executar” e ver os valores finais após a execução de todas as instruções. No caso do código fornecido, o valor final dos registradores pode ser visto na Figura 6. O registrador R1 armazena o valor 37 em hexadecimal (55 em decimal), que de fato corresponde ao décimo número da sequência de Fibonacci.

Registradores	
R0	0x00000037
R1	0x00000037
R2	0x00000015
R3	0x00000022
R4	0x00000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13	SP (Stack Pointer) 0x00000000
R14	LR (Link Register) 0x00000000
R15	PC (Program Counter) 0x0000000F

Figura 6: Valor final dos registradores exibidos na plataforma após execução do código

5. Melhorias futuras

Uma das principais melhorias futuras para o ARMLator é a expansão do conjunto de instruções suportadas. Atualmente, o simulador cobre um núcleo essencial de instruções da arquitetura ARM, mas ainda há várias operações relevantes ausentes, como chamadas de sub-rotina (BL, BX), instruções com múltiplos registradores (PUSH, POP), ou modos mais avançados de endereçamento. Incluir essas instruções tornaria o simulador mais completo e permitiria simular algoritmos mais sofisticados.

Outra melhoria seria implementar a exportação dos resultados de execução. Atualmente, os valores dos registradores, flags e memória são exibidos apenas no console. Permitir que o usuário salve esses dados em arquivos .json ou .csv facilitaria revisões posteriores, correções por professores ou integração com outras ferramentas de análise.

Por fim, outras ideias incluem a criação de um histórico de execução, permitindo que o usuário visualize todas as mudanças feitas nos registradores e memória ao longo do programa, bem como um sistema de detecção e sugestão de correções para erros comuns no código Assembly, ajudando o usuário a entender onde errou e como consertar.

6. Conclusão

O desenvolvimento do ARMLator permitiu aplicar na prática conceitos de arquitetura ARM que normalmente são apenas teóricos nas aulas. Criar a comunicação entre o backend em C++, o servidor Node.js e o frontend em React mostrou como diferentes camadas de uma aplicação podem trabalhar juntas para simular o funcionamento real de um processador. Durante os testes, como o cálculo da sequência de Fibonacci, foi possível perceber como cada instrução altera registradores, memória e flags, tornando visível o comportamento interno da CPU de forma didática.

Apesar de já ser funcional, o simulador ainda pode evoluir bastante. Incluir instruções de sub-rotina, manipulação de pilha e modos avançados de endereçamento tornaria possível simular programas mais complexos. Além disso, recursos como exportar os resultados ou registrar todo o histórico de execução ajudariam no estudo e na análise dos programas. No geral, o ARMLator cumpriu seu objetivo de aproximar a teoria da prática, oferecendo uma ferramenta interativa que facilita o aprendizado de conceitos centrais da arquitetura ARM.